

2.1 — Design Effective Tool Interfaces with Clear Descriptions and Boundaries

Concept

Tool descriptions are the LLM's only guide. Claude has no access to your implementation, your intent, or your mental model of the system. When deciding which tool to call — and how to call it — Claude reads only the name, description, and parameter schema you provide. Design them like API documentation for someone who cannot ask follow-up questions.

Descriptions as a Selection Mechanism

Claude selects tools by matching the user's goal against the semantic content of each description. A vague description causes Claude to guess. A precise description causes Claude to select correctly every time.

The description must answer three questions unambiguously:

1. **What does this tool do?** (its purpose, not its implementation)
2. **When should this tool be used?** (the condition or trigger)
3. **What does this tool NOT do?** (its explicit boundary)

The third question is often omitted and is the most important for disambiguation.

Writing Disambiguating Descriptions

When two tools handle related but distinct concerns, their descriptions must make the distinction explicit — even if it feels redundant.

Bad (ambiguous):

```
server.tool("get_user", "Get user information", schema, handler);
server.tool("find_user", "Find a user", schema, handler);
```

Good (explicit boundary):

```
server.tool(
  "get_user_by_id",
  "Retrieve a single user record by their exact UUID. Use this when you already have the user's",
  schema,
  handler
);
```

```
server.tool(
  "search_users",
  "Search users by name, email, or role. Returns a list of matching users. Use this when you ha
  schema,
  handler
);
```

Splitting vs. Consolidating Tools

Split one tool into multiple when:

- Two actions have different preconditions or risk profiles (read vs. write)
- Conflating them requires overly complex parameter logic (`mode: "get" | "search"`)
- One action is safe to call speculatively; the other has side effects

Consolidate into one tool when:

- The operations are always performed together with no meaningful independent use
- Splitting creates artificial back-and-forth (two round-trips to accomplish one logical step)
- The distinction would only confuse routing without improving precision

A `mode` parameter that switches between fundamentally different behaviors is a strong signal the tool should be split.

Parameter Naming and Schema Design

Parameter names and descriptions extend the tool's surface area for disambiguation. Treat each parameter description as a contract:

- Name parameters for their semantic role, not their data type (`userId` not `id` , `startDate` not `date1`)
- In the description, state the expected format explicitly: `"ISO 8601 date string, e.g. 2024-01-15"`
- Mark optional parameters and explain their effect when omitted
- Enumerate valid values inline when the set is small and fixed

Code Example

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";

const server = new McpServer({ name: "billing", version: "1.0.0" });

// GOOD: split tools with explicit boundaries and disambiguating descriptions

server.tool(
  "get_invoice",
  "Retrieve a single invoice by its exact invoice ID (format: INV-XXXXXX). " +
```

```

    "Returns full invoice details including line items and payment status. " +
    "Use this when you already have the invoice ID. " +
    "Do NOT use this to list invoices or search by customer.",
  {
    invoiceId: z
      .string()
      .describe("The invoice ID in the format INV-XXXXXX, e.g. INV-004821"),
  },
  async ({ invoiceId }) => {
    const invoice = await fetchInvoice(invoiceId);
    return { content: [{ type: "text", text: JSON.stringify(invoice) }] };
  }
);

server.tool(
  "list_customer_invoices",
  "List all invoices for a specific customer, optionally filtered by status. " +
  "Returns a summary list (ID, date, amount, status) – NOT full line items. " +
  "Use this to discover invoice IDs before calling get_invoice. " +
  "Do NOT use this to retrieve details of a single known invoice.",
  {
    customerId: z.string().describe("The customer's UUID"),
    status: z
      .enum(["paid", "unpaid", "overdue", "all"])
      .optional()
      .describe(
        "Filter by payment status. Defaults to 'all' when omitted."
      ),
  },
  async ({ customerId, status = "all" }) => {
    const invoices = await listInvoices(customerId, status);
    return { content: [{ type: "text", text: JSON.stringify(invoices) }] };
  }
);

```

Common Mistakes

1. **Omitting the boundary** — Describing only what a tool does, never what it doesn't do. Claude will use the closest-sounding tool when ambiguous; explicit "Do NOT use this for X" statements prevent misrouting.
2. **Using a `mode` parameter as a tool splitter** — A parameter like `action: "get" | "list" | "delete"` turns one tool into three with a single confusing entry point. Split them; each action gets its own description and its own selection logic.
3. **Generic names with no description** — A tool named `process_data` with the description "Processes data" gives Claude nothing to select on. Every word in the name and description is signal.

4. **Leaking implementation details** — Descriptions like "calls the `/v2/invoices/{id}` endpoint" describe the mechanism, not the purpose. Claude needs semantic meaning, not internals.
5. **Over-consolidating for convenience** — Combining read and write operations into one tool because they touch the same resource. Claude cannot reliably distinguish safe from destructive calls when they share a description.

References

- [Anthropic: Tool use overview](#)
- [MCP SDK: `server.tool\(\)` reference](#)
- [Anthropic: Best practices for tool definitions](#)